

# METAGPT：元编程

## 多代理协作框架

Sirui Hong<sup>1,\*</sup>, Mingchen Zhuge<sup>2\*</sup>, Jonathan Chen<sup>1</sup>, Xiawu Zheng<sup>3</sup>, Yuheng Cheng<sup>4</sup>, Ceyao Zhang<sup>4</sup>,  
Jinlin Wang<sup>(1)</sup>, Zili Wang, Steven Ka Shing Yau<sup>5</sup>, Zijuan Lin<sup>4</sup>,  
Liyang Zhou<sup>6</sup>, Chenyu Ran<sup>1</sup>, Lingfeng Xiao<sup>(1)(6)(7)</sup>, Chenglin Wu<sup>(1)(7)</sup>, Jürgen Schmidhuber<sup>(2)(6)(8)</sup>

<sup>1</sup>DeepWisdom, <sup>2</sup>AI Initiative, 阿卜杜拉国王科技大学, <sup>3</sup>厦门大学, <sup>4</sup>深圳香港中文大学

, <sup>5</sup>南京大学, <sup>6</sup>宾夕法尼亚大学,

<sup>7</sup>加州大学伯克利分校,

<sup>8</sup>瑞士人工智能实验室 IDSIA/USI/SUPSI

### 摘要

通过基于大型语言模型（LLM）的代理系统自动解决的问题的工作已经取得了显著进展。现有的基于 LLM 的多代理系统已经可以解决简单的对话任务。然而，由于天真地将 LLM 串联起来会产生层叠幻觉，导致逻辑不一致，从而使更复杂任务的解决方案变得复杂。在此，我们介绍一种创新的元编程框架--MetaGPT，它将高效的人类工作流程融入基于 LLM 的多机器人协作中。MetaGPT 将标准化操作程序（SOP）编码为提示序列，以简化工作流程，从而让具有类似人类领域专业知识的代理验证中间结果并减少错误。MetaGPT 利用流水线模式为不同的代理分配不同的角色，有效地将复杂的任务分解为涉及多个代理协同工作的子任务。与以前基于聊天的多代理系统相比，MetaGPT 在集体软件工程基准测试中生成的解决方案更加协调一致。我们的项目可在 <https://github.com/geekan/MetaGPT> 上找到。

## 1 引言

利用大型语言模型（LLM）的自主代理为加强和复制人类工作流程提供了大有可为的机会。然而，在现实世界的应用中，现有系统（Park 等人，2023 年；Zhuge 等人Cai，20232023 年c 年；等等Li 等人Wang，年年；人人Du，；，2023；等Liang 等人人人Hao 等，2023 年2023 年2023；，；，年）往往过于简化复杂性。他们难以实现有效、连贯和准确的问题解决过程，尤其是在需要有意义的协作互动时（Chen 等人，2024；Zhang Zhou 等人Dong 等人，2023；，2023；等，20232023；Qian 等人，）。

通过广泛的合作实践，人类已经在各个领域制定了广为接受的标准化操作程序（SOPs）（Belbin, 2012; Manifesto, 2001; DeMarco & Lister, 2013）。这些 SOP 在支持任务分解和有效协作方面发挥着至关重要的作用。此外，标准作业程序还概括了每个团队成员的职责，同时建立了中间产出的标准。定义明确的 SOP 可提高任务执行的一致性和准确性，使其符合规定的角色和质量标准（Belbin, 2012；Manifesto, 2001；DeMarco & Lister, 2013；Wooldridge & Jennings, 1998）。例如，在一家软件公司，产品经理通过分析竞争和用户需求，使用标准化结构创建产品需求文档（PRD），以指导开发流程。

受这些想法的启发，我们设计了一种基于 GPT 的元编程框架，名为 MetaGPT，它能从 SOP 中获得显著的益处。与其他作品（Li 等人，2023；Qian 等人，2023）不同的是，MetaGPT 要求代理生成结构化输出，如高质量需求

\*这些作者对本研究做出了同等贡献。

<sup>†</sup>吴成林 (alexanderwu@fuzhi.ai) 为通讯作者，隶属于 DeepWisdom。

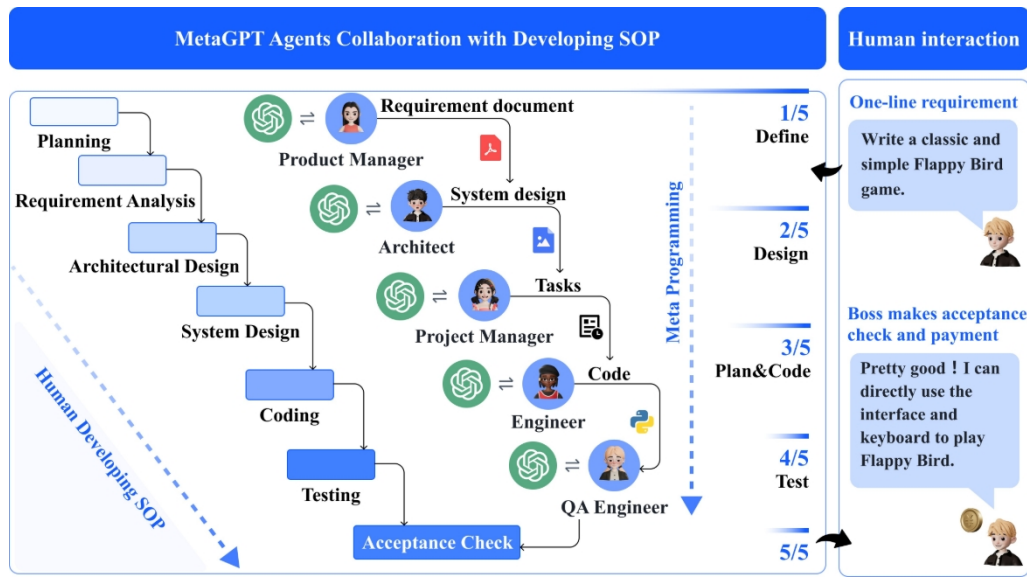


图 1: MetaGPT 与现实世界中人类团队之间的软件开发 SOP。在软件工程中, SOP 可促进不同角色之间的协作。MetaGPT 展示了其将复杂任务分解为分配给不同角色(如产品经理、架构师、工程师等)的具体可操作程序的能力。

这些输出包括文档、设计工件、流程图和接口规范。使用中间结构输出可显著提高目标代码生成的成功率。因为它有助于保持沟通的一致性,最大限度地减少协作过程中的歧义和错误。更形象地说,在 MetaGPT 模拟的公司中,所有员工都遵循严格的流水线工作流程,所有交接工作都必须符合某些既定标准。这就降低了因法律硕士之间闲聊而产生幻觉的风险,尤其是在角色扮演的框架下,例如:“嗨,你好,你好吗?”“嗨,你好吗? - 爱丽丝(产品经理);“很好!你吃过午饭了吗? - 鲍勃(建筑师)。

得益于 SOP, MetaGPT 为元编程提供了一种前景广阔的方法。在这里,我们将元编程<sup>1</sup>定义为“为编程而编程”,这与更广泛的元学习和“为学习而学习”(Schmidhuber, 1987; 1993a; Hochreiter 等人, 2001; Schmidhuber, 2006; Finn 等人, 2017) 形成鲜明对比。

这种元编程的概念还包括 CodeBERT (Feng 等人, 2020 年) 等早期工作以及 CodeLlama (Roziere 等人, 2023 年) 和 WizardCoder (Luo 等人, 2023 年) 等近期项目。然而, MetaGPT 作为一种独特的解决方案脱颖而出,通过组织良好的专业代理群实现了高效的元编程。每个代理都有特定的角色和专长,并遵循一些既定的标准。这样,就可以在运行期间自动进行需求分析、系统设计、代码生成、修改、执行和调试,从而突出基于代理的技术如何增强元编程。

为了验证 MetaGPT 的设计,我们使用公开的 HumanEval (Chen 等人, 2021a) 和 MBPP (Austin 等人, 2021) 进行评估。值得注意的是,在代码生成基准测试中, MetaGPT 达到了新的先进水平 (SoTA), Pass@1 的测试结果为 85.9% 和 87.7%。与其他用于创建复杂软件项目的流行框架相比,如 AutoGPT (Torantulino 等人, 2023 年)、LangChain (Chase, 2022 年)、AgentVerse (陈等人, 2023 年年) 和 ChatDev (钱等人, 2023)。MetaGPT 在处理更高水平的软件复杂性和提供广泛功能方面也表现突出。值得注意的是,在我们的实验评估中, MetaGPT 实现了 100% 的任务完成率,证明了我们设计的稳健性和效率(时间和令牌成本)。

我们将我们的贡献总结如下:

<sup>1</sup> <https://en.wikipedia.org/w/index.php?title=Metaprogramming>

- 我们介绍了基于 LLM 的多代理协作元编程框架 MetaGPT。它非常方便灵活，具有定义明确的角色和信息共享等功能，是开发基于 LLM 的多代理系统的有用平台。
- 我们在 MetaGPT 的设计中创新性地集成了类人 SOP，大大提高了其鲁棒性，减少了基于 LLM 的代理之间的无益协作。此外，我们还引入了一种新颖的执行反馈机制，可在运行期间调试和执行代码，大大提高了代码生成质量（例如，与 MBPP 相比，绝对值提高了 5.4%）。
- 我们在 HumanEval（陈等人，2021a）和 MBPP（奥斯汀等人，2021）上取得了最先进的性能。大量结果令人信服地验证了 MetaGPT，表明它是开发基于 LLM 的多智能体系统的一个前景广阔的元编程框架。

## 2 相关工作

自动编程 自动编程的起源可以追溯到上个世纪。1969 年，Waldinger 和 Lee（1969 年）推出了 "PROW" 系统，该系统旨在接受用谓词微积分编写的程序说明、生成算法并创建 LISP 实现（McCarthy，1978 年）。Balzer（1985 年）和 Soloway（1986 年）努力推进自动编程，并确定了实现自动编程的潜在方法。最近的方法使用了自然语言处理（NLPNi 等人）技术（，2023 年；Skreta 人等Feng，2023 年；等等人人Chen 等人，2020 年；Li，2022 年；，2018 年；2021b；Zhang 等人，2023 年）。自动编程已发展成为一个提供付费功能的行业，如微软的 Copilot。最近，基于 LLMs 的代理（Yao 等人，2022；Shinn 等人，20232023；Lin 等人，）推动了自动编程的发展。其中，ReAct（Yao等人，2022年）和Reflexion（Shinn等人，2023年2022年）利用一连串的思维提示（Wei等人，）来生成具有 LLMs 的推理轨迹和行动计划。这两项工作都证明了 ReAct 式推理循环作为授权自动编程设计范例的有效性。此外，ToolFormer（Schick 等人，2023 年）可以通过简单的 API 学习如何使用外部工具。李等人（2023 年）的研究与我们的工作最为接近，他们提出了一个直接的角色扮演编程框架，涉及扮演不同角色的代理之间的交流。Qian 等人（2023 年）利用多个代理进行软件开发。尽管现有论文（Li 等人，2023；Qian 等人，2023）提高了工作效率，但它们并没有充分挖掘具有结构化输出格式的有效工作流程。这就增加了处理复杂软件工程问题的难度。

**基于 LLM 的多代理框架** 最近，基于 LLM 的自主代理在工业界和学术界都引起了极大的兴趣（Wang 等，2023b）。许多研究（Chen 等人，2024；Wang 等人，2023c；Du 等人，2023；Zhuge 等人，20232023；Hao 等人，；Akata 等人，2023）通过整合多个代理之间的讨论，提高了 LLM 解决问题的能力。Stable-Alignment（Liu 等人，2023 年）通过在沙盒中与 LLM 代理互动，就价值判断达成共识，从而创建了指令数据集。其他作品则关注社会学现象。例如，Generative Agents（Park 等人，2023 年）创建了一个由 25 个代理组成的 "小镇"，以研究语言交互、社会理解和集体记忆。在基于自然语言的心智社会（NLSOM）（诸葛等人，2023 年）中，具有不同功能的代理通过多轮 "头脑风暴" 进行互动，以解决复杂的任务。Cai 等人（2023 年）提出了一种通过将大型模型作为工具制造者和小型模型作为工具使用者相结合来降低成本的模式。

一些作品强调与规划和战略相关的合作与竞争（Bakhtin等人，2022年）；另一些作品则提出了基于 LLM 的经济（诸葛等人，2023年）。这些著作侧重于开放世界的人类行为模拟，而 MetaGPT 则旨在将人类实践引入多代理框架。此外，基于 LLM 的代理还面临着 "助理重复指令" 或 "信息无限循环" 的挑战（Talebirad & Nadiri, 2023; Li et al.）在以任务为导向的合作中，这些挑战变得更加紧迫，因为这种合作需要持续和互利的互动（Elazar 等人，2021 年；Wang 等人Jiang 等人，2022 年；，2023 年）。这促使我们将软件开发中的标准操作程序等先进概念应用到多机器人框架中。

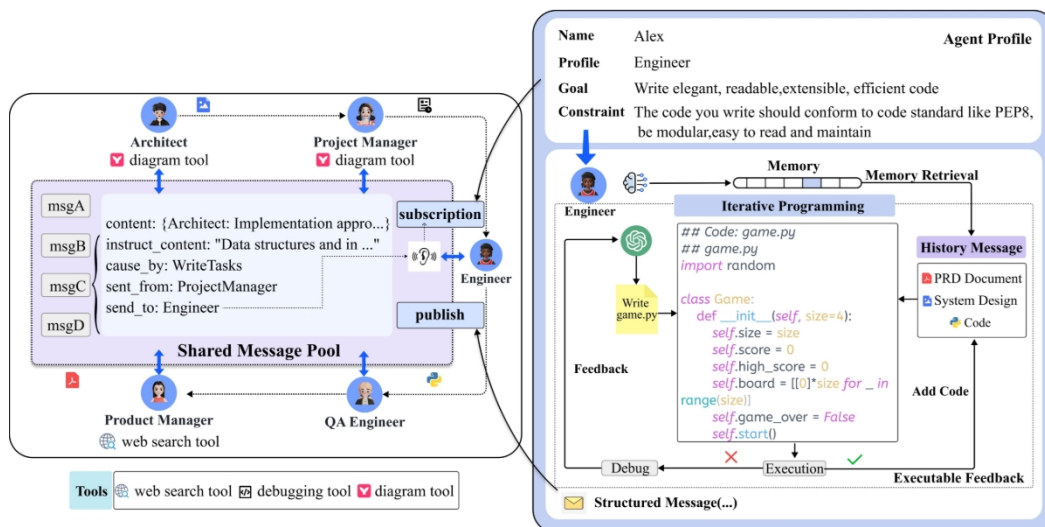


图 2: 通信协议（左）和可验证反馈的迭代编程（右）示例。**左图**：代理使用共享消息池发布结构化消息。它们还可以根据自己的配置文件订阅相关信息。右图生成初始代码后，工程师代理运行并检查错误。如果出现错误，代理会检查内存中存储的过往信息，并将其与 PRD、系统设计和代码文件进行比较。

### 3 元编程框架元编程框架

MetaGPT 是基于 LLM 的多代理系统的元编程框架。第 3.1 节解释了该框架中的角色专业化、工作流和结构化通信，并说明了如何在 SOP 的背景下组织多代理系统。第 3.2 节介绍了一种能提高角色通信效率的通信协议。我们还实现了结构化通信接口和有效的发布-订阅机制。通过这些方法，代理可以从其他角色那里获取方向信息，并从环境中获取公共信息。最后，我们在第 3.3 节中介绍了可执行反馈--一种在运行期间进一步提高代码生成质量的自我纠正机制。

#### 3.1 标准操作程序中的代理

**角色专业化** 明确的角色专业化可以将复杂的工作分解成更小、更具体的任务。要解决复杂的任务或问题，往往需要具有不同技能和专业知识的代理人员相互协作，每个人都要针对具体问题提供专门的产出。

在软件公司中，产品经理通常负责面向业务的分析并提出见解，而软件工程师则负责编程。我们在软件公司中定义了五种角色：产品经理、架构师、项目经理、工程师和 QA 工程师，如图 1 所示。在 MetaGPT 中，我们指定了代理的配置文件，其中包括他们的姓名、配置文件、目标和每个角色的约束条件。我们还为每个角色初始化了特定的上下文和技能。例如，产品经理可以使用网络搜索工具，而工程师则可以执行代码，如图 2 所示。所有代理都遵循 Yao 等人（2022 年）所描述的 React 风格行为。

每个代理都会监控环境（即 MetaGPT 中的消息池），以发现重要的任务（如来自其他代理的消息）。这些信息可以直接触发行动，也可以协助完成任务。

**跨代理的工作流程** 通过定义代理的角色和操作技能，我们可以建立基本的工作流程。在我们的工作中，我们遵循软件开发中的 SOP，使所有代理都能按部就班地工作。



图 3: MetaGPT 软件开发流程示意图, 强调其对 SOP 的重要依赖性。更详细的演示见附录 B。

具体来说, 如图 1 所示, 在获得用户需求后, 产品经理会进行全面分析, 制定详细的 PRD, 其中包括用户故事和需求池。这就是初步的功能分解。然后, 结构化的 PRD 交给架构师, 由他将需求转化为系统设计组件, 如文件列表、数据结构和接口定义。在系统设计中捕捉到信息后, 这些信息就会转给项目经理进行任务分配。工程师开始执行所设计的类和功能 (详见图 2)。在下一阶段, 质量保证工程师将制定测试用例, 以确保严格的代码质量。最后, MetaGPT 会生成精心制作的软件解决方案。我们提供了 MetaGPT 中 SOP 工作流程的详细示意图 (图 3) 和具体实例 (附录 B)。

### 3.2 通信协议

**结构化通信接口** 当前大多数基于 LLM 的多代理框架 (Li 等人, 2023 年; Zhuge 等人 2023 年; Zhang 等人, 2023 年; Park 等人, 2023 年) 都使用无约束自然语言作为通信接口。

然而, 尽管自然语言用途广泛, 还是会出现一个问题: 纯自然语言交流是否足以解决复杂的任务? 例如, 在电话游戏 (或中文

悄悄话)<sup>2</sup>，经过几轮交流后，原始信息可能会被严重扭曲。受人类社会结构的启发，我们建议使用结构化通信来制定代理的通信方式。我们为每个角色建立了一个模式和格式，并要求个体根据其特定角色和上下文提供必要的输出。

如图 3 所示，架构师代理生成两个输出：系统界面设计和序列流程图。它们包含系统模块设计和交互顺序，是工程师的重要成果。与 ChatDev (Zhao 等人, 2023 年) 不同，MetaGPT 中的代理通过文档和图表（结构化输出）而非对话进行交流。这些文档包含所有必要信息，避免了无关内容或缺失内容。

**发布-订阅机制** 共享信息在协作中至关重要。例如，建筑师和工程师经常需要参考 PRD。然而，正如以前的工作 (Li 等人, 2023; Zhao 等人, 2023; Zhang 等人, 2023) 所指出的那样，每次都以一对一的方式交流这些信息会使通信拓扑复杂化，从而导致效率低下。

为应对这一挑战，一种可行的方法是在全局信息池中存储信息。如图 2（左）所示，我们引入了一个共享信息池，允许所有代理直接交换信息。这些代理不仅可以在池中发布自己的结构化信息，还可以透明地访问来自其他实体的信息。任何代理都可以直接从共享池中获取所需的信息，而无需询问其他代理并等待他们的回复。这就提高了通信效率。

与每个代理共享所有信息会导致信息超载。在任务执行过程中，代理通常只希望接收与任务相关的信息，避免因无关细节而分心。有效管理和传播这些信息起着至关重要的作用。我们提供了一个简单有效的解决方案--订阅机制（如图 2（左）所示）。代理不依赖对话，而是利用特定角色的兴趣来提取相关信息。他们可以根据自己的角色特征选择要关注的信息。在实际应用中，代理只有在收到所有先决条件后才会启动其行动。如图 3 所示，架构师主要关注产品经理提供的 PRD，而质量保证工程师等角色提供的文档可能关注较少。

### 3.3 可执行反馈的迭代编程

在日常编程任务中，调试和优化过程发挥着重要作用。然而，现有方法往往缺乏自我修正机制，导致代码生成不成功。之前的工作引入了不可执行代码审查和自我反省 (Zhao 等人, 2023; Yao 等人 Shinn 等人, 2022; , 2023; Dong 等人, 2023)。然而，它们在确保代码可执行性和运行时正确性方面仍面临挑战。

我们最初的 MetaGPT 实现在审查过程中由于 LLM 幻觉而忽略了某些错误 (Manakul 等人, 2023 年)。为了克服这一问题，在初始代码生成之后，我们引入了一种可执行的反馈机制来反复改进代码。更具体地说，如图 2 所示，工程师被要求根据最初的产品需求和设计编写代码。

这样，工程师就能利用自身的历史执行和调试内存，不断改进代码。为获取更多信息，工程师会编写并执行相应的单元测试案例，然后接收测试结果。如果测试结果令人满意，就会启动额外的开发任务。否则，工程师会在继续编程前调试代码。这种迭代测试过程一直持续到测试通过或达到最多三次重试为止。

## 4 实验

### 4.1 实验环境

**数据集** 我们使用了两个公共基准：HumanEval (Chen 等人, 2021a) 和 MBPP (Austin 等人, 2021)，以及一个自创的、更具挑战性的软件开发基准，名为 Soft-

<sup>2</sup> [https://en.wikipedia.org/wiki/Chinese\\_窃窃私语](https://en.wikipedia.org/wiki/Chinese_窃窃私语)

wareDev: (1) HumanEval 包括 164 个手写编程任务。这些任务包括功能说明、描述、参考代码和测试。(2) MBPP 包含 427 个 Python 任务。这些任务涵盖核心概念和标准库功能，包括说明、参考代码和自动测试。(3) 我们的 SoftwareDev 数据集收集了 70 个具有代表性的软件开发任务实例，每个实例都有自己的任务提示（见表 8）。这些任务的范围多种多样（见图 5），如迷你游戏、图像处理算法、数据可视化等。它们为真实的开发任务提供了一个强大的测试平台。与之前的数据集（Chen 等人，2021a；Austin 等人，2021）不同，SoftwareDev 侧重于工程方面。在比较中，我们随机选择了七个具有代表性的任务进行评估。

**评估指标** 对于 HumanEval 和 MBPP，我们采用（Chen 等人，2021a；Dong 等人，2023）提出的无偏版 Pass @k，来评估 top-k 的功能准确性。

生成的代码：通过 @k = E<sub>问题</sub>  $1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$

对于 SoftwareDev，我们优先考虑实际使用，并通过人工评估（A、E）或统计分析（B、C、D）来评估性能：(A) 可执行性：该指标将代码从 1（故障/无功能）到 4（无缺陷）进行评级。1 "代表无功能，"2 "代表可运行但不完善，"3 "代表接近完善，"4 "代表无缺陷。(B) 成本：这里的成本评估包括 (1) 运行时间、(2) 令牌使用情况，以及 (3) 支出。(C) 代码统计：包括 (1) 代码文件；(2) 每个文件的代码行数；(3) 总代码行数。(D) 生产率：基本定义为代币使用量除以代码行数，指每行代码消耗的代币。(E) 人工修改成本：指人工修改代码的次数，人工修改代码是为了解决软件包导入错误、类名不正确或引用路径不完整等问题。通常情况下，每次更正最多涉及 3 行代码。

**基线** 我们将我们的方法与代码生成领域最新的特定领域 LLM 进行了比较，包括 AlphaCode（Li 等，2022 年）、Incoder（Fried 等等，2022 年2023 年2023 年2021a 2022 年）、CodeGeeX（Zheng 等Chen 等年Chen 等，）、CodeGen（Nijkamp，）、CodeX（，）和 CodeT（，）、2023）、CodeGen（Nijkamp 等人，2023）、CodeX（ChenChen 等 等人，2021a）和 CodeT（人Chowdhery 等，2022），以及 PaLM（人，2022）和 GPT-4（OpenAI，2023）等通用领域 LLM。Dong 等人（2023 年）提供了一些基线结果（如 Incoder、CodeGeeX）。在 HumanEval 和 MBPP 中，我们略微修改了提示，以符合响应格式要求。这些修改旨在解决特定格式的问题（即 Python 问题）。通过 SoftwareDev 基准，我们对 MetaGPT、AutoGPT（Torantulino 等人，2023 年）、LangChain（Chase，2022 年）与 Python Read-Eval-Print Loop (REPL) 工具<sup>3</sup>、Agent-Verse（陈等人，2023 年年）和 ChatDev（钱等人，2023）进行了全面比较。

## 4.2 主要结果

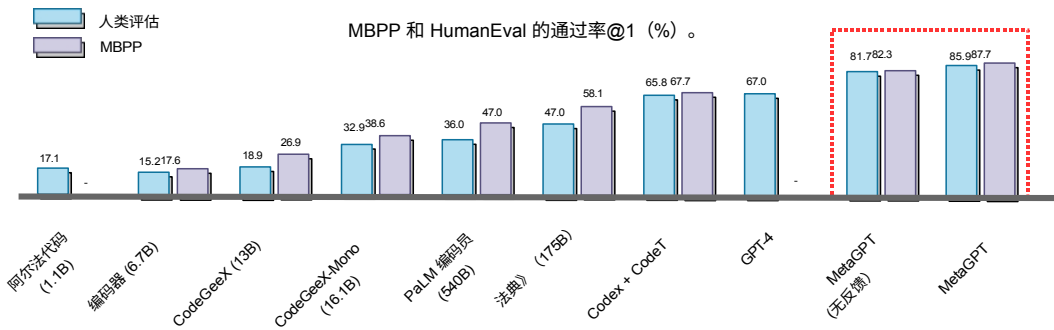


图 4：MBPP 和 HumanEval 的一次通过率。

**性能** 图4显示，在HumanEval和MBPP基准测试中，MetaGPT的性能均优于之前的所有方法。当 MetaGPT 与 GPT-4 协作时，与 GPT-4 相比，MetaGPT 在 HumanEval 基准中显著提高了通过率 @k。它分别达到了 85.9%

和 87.7% 的通过率。

---

<sup>3</sup> [https://en.wikipedia.org/wiki/Read-eval-print 循环](https://en.wikipedia.org/wiki/Read-eval-print_循环)

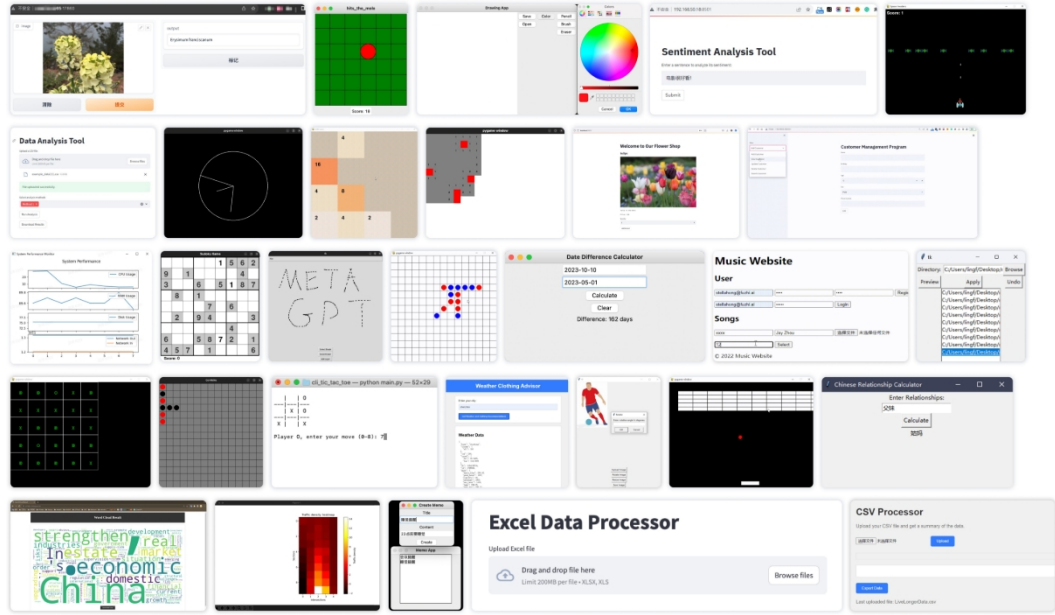


图 5: MetaGPT 开发的 演示软件。

在这两个公开基准测试中，MetaGPT 的性能均优于 ChatDev。此外，如表 1 所示，在具有挑战性的 SoftwareDev 数据集上，MetaGPT 几乎在所有指标上都优于 ChatDev。例如，在可执行性方面，MetaGPT 达到了 3.75 分，非常接近 4 分（完美）。此外，它花费的时间（503 秒）也明显少于 ChatDev。考虑到代码统计和人工审核成本，它也明显优于 ChatDev。虽然 MetaGPT 需要更多的令牌（24,613 或 31,255 个，而 ChatDev 为 19,292 个），但它只需要 126.5/124.3 个令牌就能生成一行代码。相比之下，ChatDev 使用了 248.9 个令牌。这些结果凸显了 SOP 在多代理协作中的优势。此外，我们还通过可视化示例展示了 MetaGPT 的自主软件生成能力（图 5）。有关其他实验和分析，请参阅附录 C。

表 1: 关于 SoftwareDev.

| 统计索引                   | 聊天开发工具 | 无反馈的 MetaGPT | MetaGPT |
|------------------------|--------|--------------|---------|
| (A) 可执行性               | 2.25   | 3.67         | 3.75    |
| (B) 成本#1: 运行时间 (秒)     | 762    | 503          | 541     |
| (B) 成本#2: 令牌使用量        | 19,292 | 24,613       | 31,255  |
| (C) 代码统计#1: 代码文件       | 1.9    | 4.6          | 5.1     |
| (C) 代码统计 #2: 每个文件的代码行数 | 40.8   | 42.3         | 49.3    |
| (C) 代码统计 #3: 总代码行数     | 77.5   | 194.6        | 251.4   |
| (D) 生产率                | 248.9  | 126.5        | 124.3   |
| (E) 人力修订费用             | 2.5    | 2.25         | 0.83    |

### 4.3 能力分析

与 AutoGPT 等开源基线方法以及 AgentVerse 和 ChatDev 等自主代理相比，MetaGPT 为软件工程任务提供了各种功能。如表 2 所示，我们的框架具有广泛的能力，可高效处理复杂和专业的开发任务。纳入 SOP（

如角色扮演专长、结构化交流、简化工作流程）可显著改善代码生成。其他基准方法

表 2: MetaGPT 与其他方法的能力比较。✓' 表示相应框架中存在特定功能, " 表示不存在。

| 框架能力                        | AutoGPT | LangChain | AgentVerse | ChatDev | MetaGPT |
|-----------------------------|---------|-----------|------------|---------|---------|
| 产生 PRD                      | ✓       | ✓         | ✓          | ✓       | ✓       |
| Tenical design geneneration | ✓       | ✓         | ✓          | ✓       | ✓       |
| 生成应用程序接口                    | ✓       | ✓         | ✓          | ✓       | ✓       |
| 代码生成                        | ✓       | ✓         | ✓          | ✓       | ✓       |
| 预编译执行                       | ✓       | ✓         | ✓          | ✓       | ✓       |
| 基于角色的任务管理                   | ✓       | ✓         | ✓          | ✓       | ✓       |
| 代码审查                        | ✓       | ✓         | ✓          | ✓       | ✓       |

表 3: 角色消融研究。#"表示 "数量", "产品 "表示 "产品人

ager", "Project "表示 "项目经理"。✓' 表示增加了一个特定角色。修订 "指 "人工修订成本"。

| 工程师 | 产品架构师 | 项目 | #代理 | #线路   | 开支      | 修订  | 可执行性 |
|-----|-------|----|-----|-------|---------|-----|------|
| ✓   | ✓     | ✓  | 1   | 83.0  | \$0.915 | 10  | 1.0  |
| ✓   | ✓     | ✓  | 2   | 112.0 | \$1.059 | 6.5 | 2.0  |
| ✓   | ✓     | ✓  | 3   | 143.0 | \$1.204 | 4.0 | 2.5  |
| ✓   | ✓     | ✓  | 3   | 205.0 | \$1.251 | 3.5 | 2.0  |
| ✓   | ✓     | ✓  | 4   | 191.0 | \$1.385 | 2.5 | 4.0  |

类似于在 LLM 中注入思维链 (Wei 等人, 2022 年), 可以很容易地集成类似 SOP 的设计来提高性能。

#### 4.4 消融研究

**角色的有效性** 为了了解不同角色对最终结果的影响, 我们执行了两项任务, 包括生成有效代码和计算平均统计数据。当我们排除某些角色时, 就会生成不可行的代码。如表 3 所示, 增加不同于工程师的角色后, 修订率和可执行性都得到了持续改善。虽然更多的角色会略微增加开支, 但整体性能却得到了明显改善, 这表明了各种角色的有效性。

**可执行反馈机制的效果** 如图 4 所示, 在 MetaGPT 中加入可执行反馈机制后, Hu-manEval 和 MBPP 的 Pass @1 分别显著提高了 4.2% 和 5.4%。此外, 表 1 显示, 反馈机制提高了可行性 (从 3.67 提高到 3.75), 降低了人工修改成本 (从 2.25 降低到 0.83)。这些结果说明了我们设计的反馈机制可以产生更高质量的代码。表 4 和表 9 显示了 MetaGPT 和不含可执行反馈的 MetaGPT 的其他量化结果。

## 5 结论

本作品介绍了一种新颖的元编程框架--MetaGPT, 它利用 SOP 增强了基于大型语言模型 (LLM) 的多代理系统的问题解决能力。MetaGPT 将一组代理建模为一个模拟软件公司, 类似于模拟城镇 (Park 等人, 2023 年) 和旅行者中的 Minecraft 沙盒 (Wang 等人, 2023a)。MetaGPT 利用角色专业化、工作流管理和高效共享机制 (如信息库和订阅), 使其成为自主代理和多代理框架的灵活可移植平台。它使用可执行反馈机制来提高运行时的代码生成质量。在大量实验中, MetaGPT 在多个基准测试中取得了最先进的性能。类人 SOP 的成功集成激发了未来对人工多代理系统的人类启发技术的研究。我们还将我们的工作视为规范基于 LLM 的多代理框架的早期尝试。另请参见展望 (附录 A)。

## 鸣谢

我们感谢 KAUST AI Initiative 执行秘书 Sarah Salhi 和 KAUST AI Initiative 博士后研究员王宇辉帮助润色部分文字。我们还要感谢 KAUST AI Initiative 的博士生王文怡，感谢她为论文提供了全面的反馈，并与明辰一起帮助起草了展望（附录）。我们还要感谢 DeepWisdom 副总裁徐宗泽为 AgentStore 提供的说明材料。

## 作者供稿

洪思睿主持了大部分实验，并设计了可执行的反馈模块。她还领导了最初版本的撰写工作，并得到了张才瑶、王金林和王自力的支持。诸葛明晨设计了自我改进模块，讨论了其他实验，并主持了目前的撰写工作。Jonathan Chen 协助进行了 MBPP 实验，概述了方法部分，并参与了当前的撰写工作。Xiawu Zheng 提供了宝贵的指导意见，审阅并编辑了本文。程宇恒对评估指标设计和 HumanEval 实验做出了贡献。Steven Ka Shing Yau、Zijuan Lin、Liyang Zhou、Lingfeng Xiao 协助进行了 MBPP 实验以及与开源基线方法的比较。冉晨宇绘制了大部分插图。吴成林是 DeepWisdom 的首席执行官，他是 MetaGPT 的发起人，为 MetaGPT 贡献了最重要的代码，并为本项目提供了建议。Juergen Schmidhuber 是 KAUST 的人工智能计划主任和 IDSIA 的科学主任，他为本项目提供了建议并帮助撰写了文章。

## 参考资料

Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, and Eric Schulz. 用大型语言模型玩重复游戏》，*arXiv preprint*, 2023.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le 和 Charles Sutton. 使用大型语言模型的程序合成，2021 年。

Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu 等。将语言模型与战略推理相结合，在外交游戏中进行人类水平的游戏。*科学*》，2022 年。

罗伯特-巴尔泽自动编程的 15 年展望》。TSE，1985 年。

R.M. Belbin. 工作中的团队角色》。Routledge, 2012. URL <https://books.google.co.uk/books?id=MHIQBAAQBAJ>。

蔡天乐、王学智、马腾宇、陈新云、周丹妮。作为工具制造商的大型语言模型》，*arXiv preprint*, 2023.

哈里森-蔡斯。LangChain。 <https://github.com/hwchase17/langchain>, 2022

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet：代码生成与生成测试》，2022 年。

陈家琪、姜玉贤、陆家琛、张莉。S-agents: open-ended environment 中的自组织代理。 *ArXiv preprint*, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, 默罕默德-巴伐利亚、克莱门斯-温特、菲利普-蒂莱、费利佩-彼得罗夫斯基-苏奇、戴夫-卡明斯、马蒂亚斯-普拉珀特、福-提奥斯-昌齐斯、伊丽莎白-巴恩斯、阿里尔-赫伯特-沃斯、威廉-赫伯根-古斯、亚历克斯-尼科尔、亚历克斯-帕诺、尼古拉斯-特扎克、唐杰、伊戈尔-巴布什金、苏奇尔-巴拉吉、尚塔努-詹恩、威廉-桑德斯、克里斯托弗-赫塞、安德鲁-N.Carr、Jan Leike、Josh Achiam、Vedant Misra、Evan Morikawa、Alec Radford、Matthew Knight、Miles Brundage、Mira Murati、

Katie Mayer、Peter Welinder、Bob Mc-Grew、Dario Amodei、Sam McCandlish、Ilya Sutskever 和 Wojciech Zaremba。评估基于代码训练的大型语言模型，2021a.

陈伟泽、苏裕生、左经纬、杨程、袁晨菲、钱晨、陈智敏、秦宇佳、卢雅茜、谢若冰、刘志远、孙茂松和周杰。Agentverse: 促进多代理协作和探索代理的突发行》, 2023 年。

Xinyun Chen, Chang Liu, and Dawn Song. 执行引导的神经程序合成。In *ICLR*, 2018.

Xinyun Chen, Dawn Song, and Yuandong Tian. 超越特定领域语言的神经程序合成潜执行。 *NeurIPS*, 2021b.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 掌上电脑: 利用路径扩展语言建模, 2022 年。

T. DeMarco 和 T.R. Lister. *Peopleware: 高效项目与团队*。Addison-Wesley, 2013 年。URL <https://books.google.co.uk/books?id=DVIsAQAAQBAJ>.

Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 通过 chatgpt 生成自协作代码。 *arXiv 预印本*, 2023.

杜一伦、李爽、安东尼奥-托拉尔巴、约书亚-B-特南鲍姆和伊戈尔-莫尔达奇。通过多代理辩论提高语言模型的事实性和推理能力》, 2023 年。

Yanai Elazar, Nora Kassner, Shauli Ravfogel, Abhilasha Ravichander, Eduard Hovy, Hinrich Schütze 和 Yoav Goldberg. 测量和改进预训练语言模态的一致性。 *TACL*, 2021.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. *ArXiv preprint*, 2020.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero 和 Tim Rocktäschel. Promptbreeder: *arXiv preprint*, 2023.

Chelsea Finn, Pieter Abbeel 和 Sergey Levine. 用于深度网络快速适应的模型识别元学习。In *ICML*, 2017.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer 和 Mike Lewis. 编码器: 用于代码填充和合成的生成模型。 *ArXiv 预印本*, 2022 年。

欧文-约翰-古德关于第一台超智能机器的推测。 *Adv. Comput.*, 1965.

郝睿、胡林梅、齐伟健、吴清流、张怡蕊、聂立强。聊天网络: 更多大脑, 更多智能。 *ArXiv 预印本*, 2023。

S. Hochreiter, A. S. Younger, and P. R. Conwell. 利用梯度下降学会学习。In *Lecture Notes on Comp. Intl. Conf. on Artificial Neural Networks (ICANN-2001)*, pp. 94. Springer: Berlin, Heidelberg, 2001.

Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 使用大型语言模型的自规划代码生成》, *arXiv preprint*, 2023.

李国豪、哈桑-阿贝德-阿尔-卡迪尔-哈穆德、哈尼-伊塔尼、德米特里-希兹布林、伯纳德-加内姆。骆驼：大型语言模型社会的 "心智 "探索交流代理。 *ArXiv 预印本*, 2023。

Yujia Li、David Choi、Junyoung Chung、Nate Kushman、Julian Schrittwieser、Re'mi Leblond、Tom Eccles、James Keeling、Felix Gimeno、Agustin Dal Lago 等用字母代码生成竞赛级代码。 *科学*》，2022 年

田亮、何志伟、焦文祥、王星、王艳、王瑞、杨玉久、涂兆鹏、史书明。通过多代理辩论鼓励大型语言模型中的发散思维》， *arXiv preprint*, 2023.

Bill Yuchen Lin、Yicheng Fu、Karina Yang、Prithviraj Ammanabrolu、Faeze Brahman、Shiyu Huang、Chandra Bhagavatula、Yejin Choi 和 Xiang Ren。Swiftsage：用于复杂交互任务的具有快慢思维的生成式代理。 *ArXiv 预印本*, 2023。

刘瑞波、杨瑞欣、贾晨艳、张戈、周丹妮、戴焱明、杨迪一和 Soroush Vosoughi。在模拟人类社会中训练社会对齐语言模型》， *arXiv preprint*, 2023.

罗子阳、徐灿、赵普、孙庆丰、耿秀波、胡文祥、陶崇阳、马静、林庆伟、蒋大新。Wizardcoder：用 evol-instruct 增强代码大型语言模型。 *arXiv 预印本*, 2023。

Potsawee Manakul、Adian Liusie 和 Mark JF Gales。Selfcheckgpt：用于生成式大型语言模型的零资源黑盒子语言识别检测。 *ArXiv 预印本*, 2023。

敏捷宣言。 *敏捷软件开发宣言*。美国犹他州雪鸟市，2001 年。约翰-麦卡锡。lisp 的历史。 *编程语言史*》。1978.

Ansong Ni、Srinu Iyer、Dragomir Radev、Veselin Stoyanov、Wen-tau Yih、Sida Wang 和 Xi Victoria Lin。杠杆：学习验证语言到代码生成的执行。 In *ICML*, 2023.

Erik Nijkamp、Bo Pang、Hiroaki Hayashi、Lifu Tu、Huan Wang、Yingbo Zhou、Silvio Savarese 和 Caiming Xiong。代码源：多转程序合成代码的开放式大型语言模型，2023 年。

OpenAI.Gpt-4 技术报告，2023 年。

Joon Sung Park、Joseph C O'Brien、Carrie J Cai、Meredith Ringel Morris、Percy Liang 和 Michael S Bernstein。生成代理：人类行为的交互式模拟。 *ArXiv 预印本*, 2023。

Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun.用于软件开发的交流型代理》，2023 年。

Baptiste Roziere、Jonas Gehring、Fabian Gloeckle、Sten Sootla、Itai Gat、Xiaoqing Ellen Tan、Yossi Adi、Jingyu Liu、Tal Remez、Je'remy Rapin 等。Code llama：代码的开放基础模型。 *ArXiv 预印本*, 2023。

Timo Schick、Jane Dwivedi-Yu、Roberto Dess'i、Roberta Raileanu、Maria Lomeli、Luke Zettlemoyer、Nicola Cancedda 和 Thomas Scialom。Toolformer：语言模型可以自学使用工具。 *ArXiv 预印本*, 2023。

J.Schmidhuber.自反权重矩阵。 In *Proceedings of the International Conference on Artificial Neural Networks, Amsterdam*, pp.Springer, 1993a.

J.Schmidhuber.Goedel machines: self-referential universal problem solvers making provably optimal selfimprovements. 技术报告 IDSIA-19-03, arXiv:cs.LO/0309048 v3, IDSIA, Manno-Lugano, Switzerland, December 2003.

J.Schmidhuber.Goedel 机器：完全自我推论的最优通用自我改进器。In B. Goertzel and C. Pennachin (eds.), *Artificial General Intelligence*, pp.Springer Verlag, 2006.variant available as arXiv:cs.LO/0309048.

- J.Schmidhuber.终极认知 *a`la Go`del. 认知计算* , 1 (2) : 177-193, 2009.
- Ju`rgen Schmidhuber. *自我参照学习中的进化原则, 或学习如何学习: 元元钩子*. 博士论文, 1987 年。
- Ju`rgen Schmidhuber. 一种 "自引用" 权重矩阵. In *ICANN'93: 国际人工神经网络会议论文集, 荷兰阿姆斯特丹*, 1993 年9月13-16日3, 1993b.
- Ju`rgen Schmidhuber. 关于学会思考: 强化学习控制器与递归神经世界模型新组合的算法信息论》。 *ArXiv preprint*, 2015.
- Ju`rgen Schmidhuber, Jieyu Zhao, and Nicol N Schraudolph. 自修正策略的强化学习见《学会学习》。1998.
- Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *ArXiv preprint*, 2023.
- Marta Skreta、Naruki Yoshikawa、Sebastian Arellano-Rubach、Zhi Ji、Lasse Bjørn Kristensen、Kourosh Darvish、Ala`n Aspuru-Guzik、Florian Shkurti 和 Animesh Garg. 错误是有用的提示: 使用验证器辅助迭代提示的指令引导任务编程》。 *ArXiv 预印本*, 2023。
- 埃利奥特·索洛维学习编程= 学习构建机制和解释。 *ACM 通信*, 1986 年。
- Yashar Talebirad 和 Amirhossein Nadiri. 多代理协作: 利用智能 IIm 代理的力量》, 2023 年。
- Torantulino et al. Auto-gpt. <https://github.com/Significant-Gravitas/Auto-GPT>, 2023.
- R.J. Waldinger 和 R. C. T. Lee. PROW: 迈向自动程序编写的一步。见 D. E. Walker 和 L. M. Norton (编), 《第一届国际人工智能联合会议 (IJCAI) 论文集》, 1969 年。
- 王冠之、谢雨琦、蒋云帆、阿杰-曼德勒卡、肖超伟、朱昱珂、范林熙和阿尼玛-阿南德库马尔。旅行者具有大型语言模型的开放式嵌入式代理。 *ArXiv 预印本*, 2023a。
- 王磊、马辰、冯雪阳、张泽宇、杨浩、张景森、陈志远、唐家凯、陈旭、林彦凯等。基于大型语言模型的自主代理研究。 *ArXiv 预印本*, 2023b。
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 自洽性改善语言模型中的思维链推理》, *arXiv preprint*, 2022.
- 王振海龙、毛绍光、吴文山、葛涛、魏福如、季恒。大型语言模型中的认知协同: 通过多角色自我协作的任务解决代理》。 *ArXiv 预印本*, 2023c。
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.
- Michael Wooldridge 和 Nicholas R. Jennings. 面向代理开发的陷阱。 *第二届自主代理国际会议论文集*, 1998 年。URL <https://doi.org/10.1145/280765.280867>.
- 姚顺禹、赵杰夫、于琰、杜楠、Izhak Shafran、Karthik Narasimhan 和 Yuan Cao. React: 语言模型中推理与行动的协同。 *ArXiv 预印本*, 2022 年。
- Eric Zelikman、Eliaana Lorch、Lester Mackey 和 Adam Tauman Kalai. 自学优化器 (停止): 递归自我改进代码生成。 *ArXiv 预印本*, 2023。

Hongxin Zhang, Weihua Du, Jiaming Shan, Qinhong Zhou, Yilun Du, Joshua B Tenenbaum, Tian- min Shu, and Chuang Gan.用大型语言模型模块化构建合作型化身代理》, *arXiv preprint*, 2023.

Xufeng Zhao, Mengdi Li, Cornelius Weber, Muhammad Burhan Hafez, and Stefan Wermter.与环境聊天：使用大型语言模型的交互式多模态感知。 *ArXiv 预印本*, 2023。

Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang.Codegeex：预训练的代码生成模型，在Humaneval-x上进行多语言评估，2023年。

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena：用于构建自主代理的现实网络环境》, *arXiv preprint*, 2023.

诸葛明辰、刘昊哲、Francesco Faccio、Dylan R Ashley、Ro’bert Csorda’s、Anand Gopalakrishnan、Abdullah Hamdi、Hasan Abed Al Kader Hammoud、Vincent Herrmann、Kazuki Irie 等。基于自然语言的心灵社会中的心灵风暴。 *ARXiv 预印本*, 2023。

## A 展望

### A.1 自我完善机制

本文正文中的 MetaGPT 版本有一个局限性，即每个软件项目都是独立执行的。然而，通过积极的团队合作，软件开发团队应从开发每个项目的过程中汲取经验，从而随着时间的推移变得更加兼容和成功。

这与递归自我改进（recursive self-improvement）的概念有些关联，递归自我改进最早于 1965 年非正式提出（Good, 1965 年），1987 年开始具体实施（Schmidhuber, 1987 年；1993b；Schmidhuber Schmidhuber 等人, 1998 年），最终形成了数学上最优的自反式自我改进器（, 2003 年；2009 年）的概念。一般来说，系统应从现实世界的经验中学习，从学习经验中元学习更好的学习算法，从元学习经验中元学习更好的元学习算法，等等，除了可计算性和物理学方面的限制外，不受任何限制。

最近的一些相关工作利用大型语言模型（LLM）的推理能力，递归改进 LLM 的提示，以提高某些下游任务的性能（Fernando 等人, 2023 年；Zelikman 等人, 2023 年），类似于 2015 年的自适应提示工程师（Schmidhuber, 2015 年），其中一个神经网络学会为另一个预先训练好的神经网络生成查询或提示序列，其答案可以帮助第一个网络更快地学习新任务。

在目前的工作中，我们还探索了一种自我参照机制，该机制可根据代理在软件开发过程中观察到的信息，递归地修改代理的连贯提示。我们的初步实施方案如下。在每个项目开始之前，软件公司的每个代理都会查看以前的反馈信息，并对他们的约束提示进行必要的调整。这使他们能够不断从过去的项目经验中吸取教训，并通过改进公司中的每个个体来增强整个多代理系统。我们首先为每个代理建立一个交接反馈行动。该行动负责认真总结以往项目开发过程中获得的信息，并将这些信息整合到更新的约束提示中。总结后的信息会存储在长期记忆中，以便在未来的约束提示更新中继承。在启动一个新项目时，每个代理都会首先做出反应。每个代理都会评估收到的反馈，并在约束提示中总结如何改进。

目前的一个局限是，这些基于摘要的优化只能修改角色专业化中的约束条件（第 3.1 节），而不能修改通信协议中的结构化通信接口（第 3.2 节）。未来的进步还有待探索。

### A.2 多代理经济体

在现实世界的团队合作中，交互流程往往不是硬编码的。例如，在一家软件公司，协作 SOP 可能会动态变化。

关于“基于自然语言的心智社会”（NLSOM）的论文（诸葛等人, 2023 年）讨论了这种自组织的一种实现方式，该论文提出了“心智经济”（EOM）的概念，这是一种用于 LLM 和其他代理的社会的强化学习（RL）框架。EOMs 不使用标准的 RL 技术，通过修改神经网络参数来优化系统的总奖励，而是利用自由市场的供求原则，将信用（金钱）分配给那些为经济成功（奖励）做出贡献的代理。

DeepWisdom 最近推出的基于代理的平台（AgentStore<sup>4</sup>）与 EOM 的信用分配概念兼容。AgentStore 中的每个代理都提供了一个服务列表，并附有相应的费用。该平台提供了便捷的应用程序接口（API），使平台中的人类用户或代理可以轻松地从其他代理那里获取服务，从而完成自己的服务。图 6 显示了 AgentStore 的用户界面（UI），其中展示了具有不同技能的各种代理。此外，个人开发者也可以参与构建新的代理，实现社区内的协作开发。具体来说，AgentStore 允许用户根据自己的需求订阅代理

<sup>4</sup> <http://beta.deepwisdom.ai>

并根据使用情况付费。此外，用户还可以购买附加功能，扩展现有代理的即插即用功能。这样，用户就可以逐步升级他们的代理。在 MetaGPT 框架内，AgentStore 可以支持各种代理的协作。用户可以将多个代理集合在一起，执行更复杂的任务或项目，所有代理共享并遵守 MetaGPT 中定义的开发和通信协议。

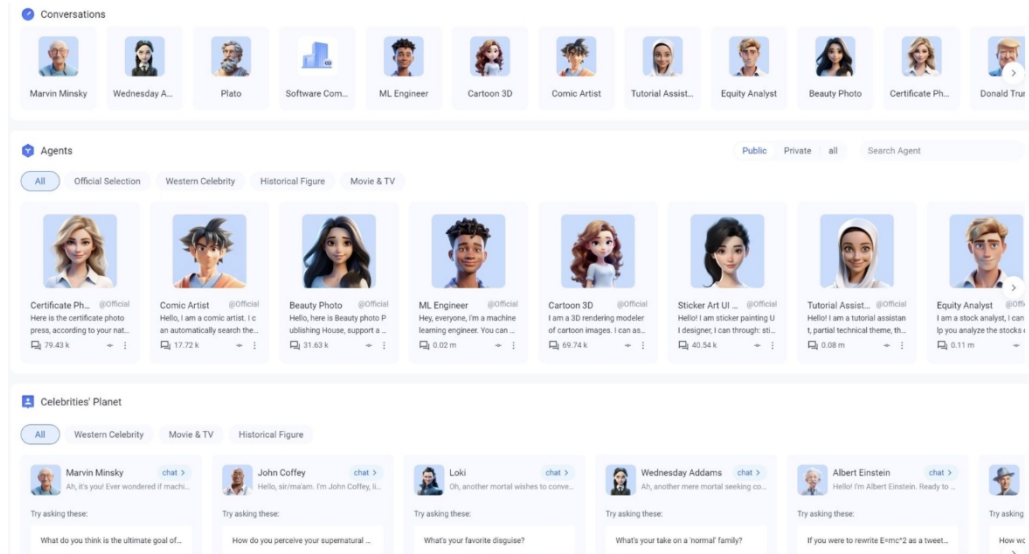


图 6：AgentStore 是一个专门为用户在 MetaGPT 框架内创建和开发代理提供服务的平台。该平台为用户提供了一个操作界面，让用户可以轻松管理各种具有不同情感、性格和能力的代理，以完成特定任务。

## B 执行演示

在本节中，我们将概述使用 MetaGPT 开发软件的全过程。它始于用户的输入命令（如附录 B.1 所示），终于根据用户的规格要求设计出的软件。

### B.1 用户输入

收到用户指令后，MetaGPT 会与专业开发团队合作完成任务。以下是用户输入的演示：

用户输入

```
1 编写一个 python3 图形用户界面应用程序，以便用它绘制图像
```

### B.2 METAGPT 开发过程

现在，我们将逐步解释每个代理的标准化输出过程。

产品经理 产品经理生成产品需求文档（PRD），详见指定文档。该文档包括目标、用户故事、竞争分析、需求分析和需求池。此外，还制作了竞争象限图（见图 7）。随后，这些文档和图表将交给架构师进行系统设计。

产品需求文件

```
1  ## 原始要求
2  老板需要一个 Python3 GUI 色彩测量仪，在移动时输出 RGB 值。
3
4  ## 产品目标
5  蟒蛇
6  [
7      "创建用户友好的图形用户界面色彩测量仪"、
8      "确保色度计准确输出 RGB 值"、
9      "确保色度计在移动过程中实时更新 RGB 值"
10 ]
11 '''
12
13 ## 用户故事
14 蟒蛇
15 [
16     "作为用户，我想选择屏幕上的任何一种颜色，以便获得其 RGB 值"、
17     "作为用户，我希望 RGB 值能在我移动颜色表时实时更新，这样我就能
18         → 可以获得准确的色彩信息"、
19     "作为用户，我希望有一个简单直观的界面，这样我就能轻松使用色度计"。
20 ]
21 '''
22
23 ## 竞争分析
24 蟒蛇
25 [
26     "Color Cop: 一款适用于 Windows 的流行拾色器。它简单易用，但缺乏真实的色彩。
27         → 时间 RGB 值更新"、
28     "Just Color Picker: 提供实时颜色选择和多种颜色代码格式。不过
29         → 其界面杂乱无章，不太方便用户使用"、
30     "即时滴管: 一个安装在系统托盘中的简单拾色器。它很方便，但
31         → 缺乏先进功能"、
32     "ColorZilla: 浏览器的拾色器扩展。它方便网页设计，但不
33         → 适合桌面应用"、
34     "ColorPic: 提供调色板和混合工具。功能丰富，但可能会让人不知所措
35         → 用于简单任务"
36 ]
37 '''
```

```

1
2 ## 需求分析
3 该产品应是一个 Python3 图形用户界面应用程序，可用作颜色测量仪。它应允许
    ↳ 用户可以选择屏幕上的任何颜色，并显示相应的 RGB 值。RGB
    ↳ 数值应随着色度计的移动而实时更新。界面应
    ↳ 简单易用。
4
5 ## Requirement Pool
6 蟒蛇
7 [
8     (为颜色测量仪设计用户友好的图形用户界面", "P0")、
9     ("实现颜色选择功能", "P0")、
10    (显示所选颜色的 RGB 值", "P0")、
11    (随着色度计的移动实时更新 RGB 值", "P0")、
12    ("测试应用程序的准确性和性能", "P1")
13 ]
14 '''
15
16 ## 用户界面设计草案
17 用户界面应简约、直观。它应主要包括一个颜色表，其中
    ↳ 可以是屏幕的一个放大区域，用户可以随意移动。的 RGB 值。
    ↳ 当前选择的颜色应醒目地显示出来，并在以下情况下实时更新
    ↳ 色彩表的移动。版面设计应简洁明了，重点突出
    ↳ 功能性和易用性。
18
19 ## Anything UNCLEAR
20 没有不清楚的地方。

```

Reach and engagement of campaigns

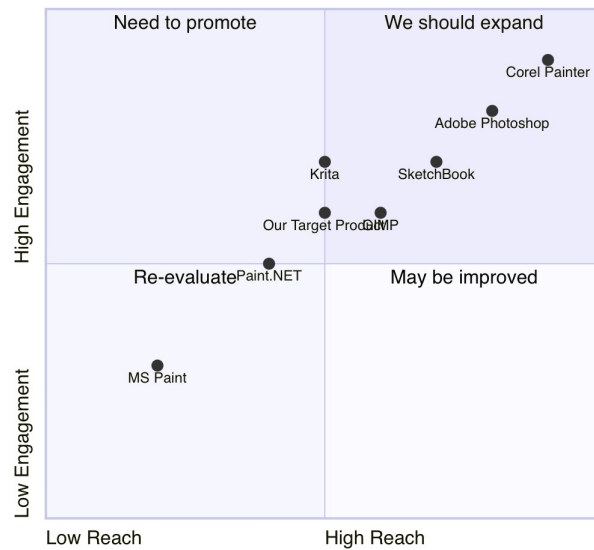


图 7：产品经理代理生成的“绘图应用程序”象限图。

架构师 根据 PRD 中的需求，架构师代理设计技术规格，包括系统架构图和接口定义。最初，架构师会定义总体技术轨迹。随后，设计项目架构，包括文件、类（图 8）和顺序流程图（图 12）。然后，将架构师的文档交给项目经理进行任务分配和执行。

系统设计

```
1 ## 实施方法
2 我们将使用 Python 的 Tkinter 库来创建颜色测量仪的图形用户界面。Tkinter 是一个标准的
   → Python 与 Tk 图形用户界面工具包的接口，大多数 Python 安装程序都包含该接口。它
   ' → 使用简单，功能强大，足以满足我们的需求。
3
4 颜色选择功能将使用 PIL (Pillow) 库实现，该库允许
   → 我们可以捕捉屏幕的像素数据。通过移动鼠标，我们可以获取以下像素的 RGB 值
   ' → 实时显示鼠标光标下的像素。
5
6 将使用 Tkinter 的标签部件在图形用户界面上显示 RGB 值。我们将使用 Python 的
   → 线程库，随着色度计的移动实时更新 RGB 值。
7
8 将使用 Python 的 unittest 库测试应用程序的准确性和性能。
9
10 ## Python 软件包名称
11 蟒蛇
12 "色度计"
13 '''
14
15 ## 文件列表
16 蟒蛇
17 [
18     "main.py",
19     "color_picker.py",
20     "gui.py",
21     "tests.py"
22 ]
23 '''
24
25 ## Anything UNCLEAR
26 对我来说，这一要求是明确的。
```

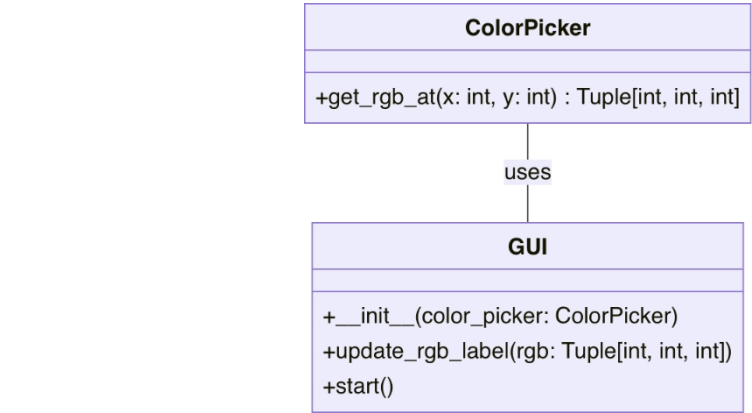


图 8： 建筑师为 "绘图应用程序 "生成的数据结构和接口定义代理。

**项目经理** 项目经理将项目分解为任务列表。此外，每个代码文件都会根据其预期功能进行分析，然后作为分配给工程师的单独任务进行处理。

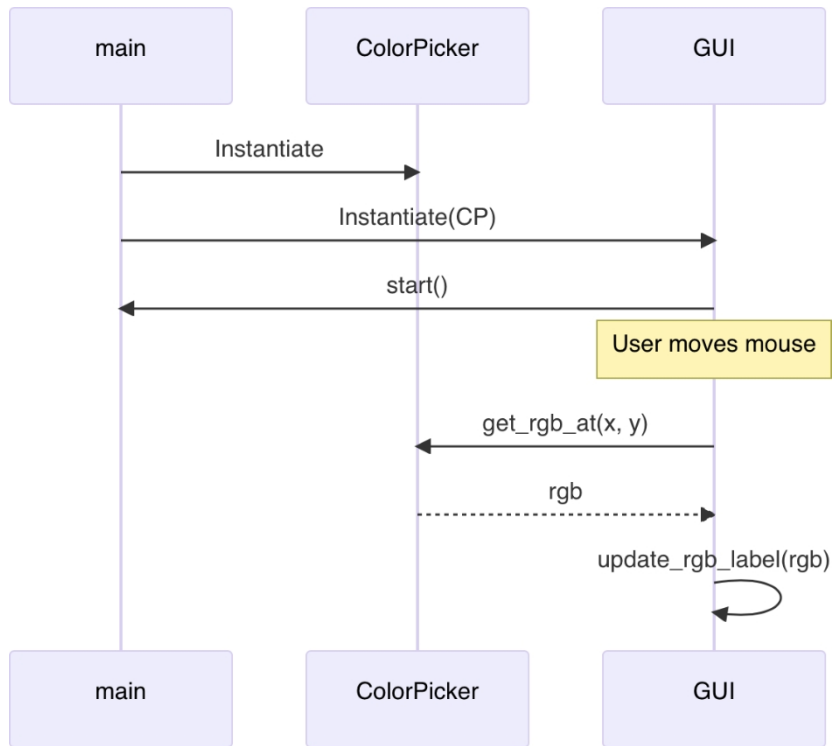


图 9：建筑师代理生成的 "绘图应用程序" 的程序调用流程。

## 任务

```

1 ## 所需 Python 第三方软件包
2
3 蟒蛇
4 """
5 tkinter==8.6
6 枕==8.3.1
7 """
8 '''
9
10 ## 所需其他语言第三方软件包
11 蟒蛇
12 """
13 无需其他语言的第三方软件包。
14 """
15 '''
16
17 ## 完整的应用程序接口规格
18 蟒蛇
19 """
20 本项目不使用应用程序接口。
21 """
22 '''
23
24 ## 逻辑分析
25 蟒蛇
26 [
27     ("main.py", "包含主程序循环并初始化所有其他组件"),
28     ("canvas.py", "执行绘图画布, 响应鼠标事件, 并使用选定的
29         → 工具和颜色"),
30     ("tools.py", "实现工具选择界面"),
31     ("color_picker.py", "实现颜色选择界面"),
32     ("file_manager.py", "实现文件保存和打开功能")
33 ]
34 '''

```

## 产品需求文件

```
1  ## 任务清单
2  蟒蛇
3  [
4      "main.py"、
5      "canvas.py"、
6      "tools.py"、
7      "color_picker.py"、
8      "file_manager.py"
9  ]
10 '''
11
12 ## 共享知识
13 蟒蛇
14  """
15  Tkinter "库用于创建应用程序的图形用户界面。它提供各种部件
16      ' → 像画框和画布一样，在这个项目中使用。
17  Pillow "库用于处理图像文件。它用于 "file_manager.py "中的
18      ' → 保存和打开图像文件。
19  main.py "文件是应用程序的入口点。它初始化所有其他组件
20      ' → 并启动应用程序循环。
21
22  """
23
24  """
25
26  ## Anything UNCLEAR
27  项目要求和实施方法是明确的。但是，我们需要确保
28      ' → 所有团队成员都熟悉 Tkinter 和 Pillow 库。如果不熟悉，他们可以
29      ' → 在开始开发之前，需要一些时间来学习这些库。
```

工程师 鉴于所提供的文件结构和函数定义，工程师代理只需具备基本的开发技能即可完成开发任务。由于文件数量庞大，我们在此只介绍一个自动生成的代码文件。

#### 生成代码 (color\_picker.py)

```
1 import tkinter as tk
2 from tkinter import colorchooser
3 class ColorPicker:
4     def __init__(self, root: tk.Tk, color: str = 'black'): self.frame
5         = tk.Frame(root)
6         self.color = color
7
8         self.color_button = tk.Button(self.frame, text='Color', command=self.select_color)
9         self.color_button.pack(fill='x')
10
11     def select_color(self):
12         color = colorchooser.askcolor()[1] 如果
13         color 不是 None:
14             self.color = color
15
16     def pack(self, **kwargs): self.frame.pack(**kwargs)
17
18     def get_color(self): 返回
19         self.color
```

质量保证工程师 在收到工程师的代码输出后，质量保证工程师会生成单元测试代码并进行审查，以识别和修复任何错误，确保软件的高质量。

#### 生成单元测试 (test.py)

```
1 导入 unittest
2 import tkinter as tk
3 # 你可能想把 ColorPicker 类移至另一个文件，比方说 color_picker.py
4 # from color_picker import ColorPicker
5
6 class TestColorPicker(unittest.TestCase):
7
8     def setUp(self): self.root =
9         tk.Tk()
10        self.cp = ColorPicker(self.root)
11
12    def tearDown(self):
13        # 这对测试后关闭 tk 窗口很重要 self.root.destroy()
14
15    def test_initial_color(self):
16        # 在初始化过程中测试默认颜色 self.assertEqual(self.cp.get_color(),
17            'black')
18
19    def test_set_and_get_color(self):
20        # 测试设置新颜色并获取它 new_color = '#ffffff' # 白色
21        self.cp.color = new_color
22        self.assertEqual(self.cp.get_color(), new_color)
23
24    名称 ==main':
25    unittest.main()
```

最终，如图 10 所示，MetaGPT 生成了一个名为 "绘图应用程序" 的功能应用程序。

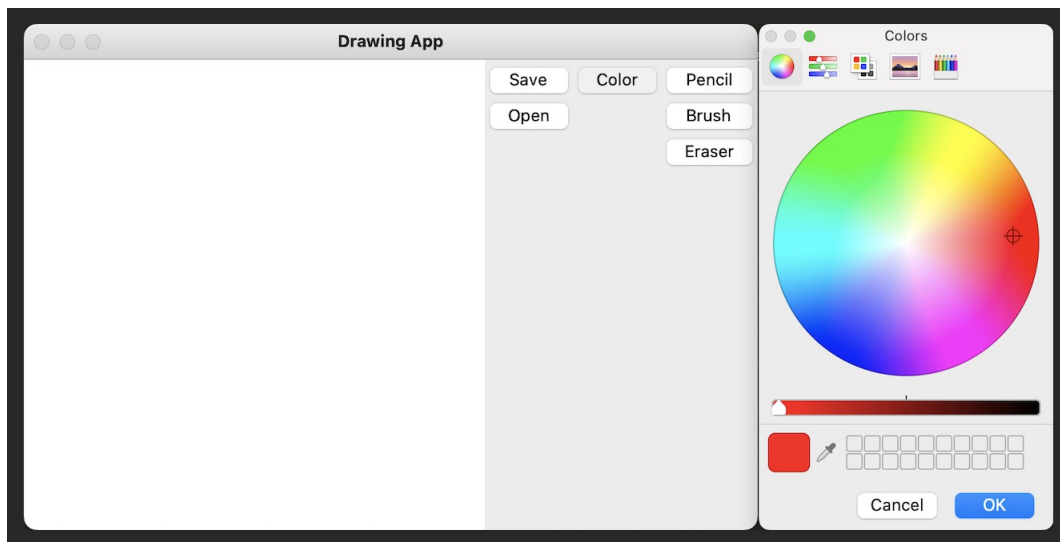


图 10: MetaGPT 生成的 "绘图应用程序"。

## C 实验

### C.1 软件开发数据集详情

SoftwareDev 数据集包括 70 个不同的软件开发任务。表 8 显示了数据集中 11 个任务的名称和详细提示。请注意，本文的主要实验使用的是所列的前七项任务。

### C.2 其他成果

**MetaGPT 的量化结果** 如表 4 所示，MetaGPT 的平均得分达到 3.9，超过了基于 Chat 链的 ChatDev 的 2.1 分 Zhao 等人 (2023 年)。对比一般智能算法的得分，包括 AutoGPT Torantulino 等人 (2023 年) 的得分均为 1.0，未能生成可执行代码。我们注意到，生成的代码往往很短，缺乏全面的逻辑，而且往往不能正确处理跨文件依赖关系。

虽然 AutoGPT (Torantulino 等人, 2023 年)、Langchain (Chase, 2022 年) 和 Agent-Verse (陈等人, 2023 年) 等模型显示出强大的一般问题解决能力，但它们缺乏开发复杂系统的基本要素：系统地解构需求。相反，MetaGPT 通过专门的分工和 SOP 工作流程，简化了将抽象需求转化为详细的类和功能设计的过程。与 Chat-Dev (Zhao 等人, 2023 年) 相比，MetaGPT 的结构化消息传递和反馈机制不仅减少了通信信息的丢失，还提高了代码的执行效率。

**无可执行文件反馈的 MetaGPT 定量结果** 表 9 显示了在软件开发项目数据集中，使用 GPT-4 32K 的 MetaGPT 在 11 项任务上的性能。它还显示了所有 70 个任务的平均性能（最后一行）。请注意，此处使用的 MetaGPT 版本是不含可执行反馈机制的基本版本。

**使用不同 LLM 的 MetaGPT 定量结果** 为了验证 MetaGPT 在不同 LLM 上的性能，我们随机选取了 5 个 SoftwareDev 任务，使用 GPT-3.5 和 Deepseek Code 33B<sup>5</sup> 作为后端进行了实验。如表 5 所示，结果表明，尽管 MetaGPT 可以使用这些 LLM 完成任务，但使用 GPT-4 作为后端会产生更优越的性能。

<sup>5</sup> <https://deepseekcoder.github.io>

表 4: **可执行性比较**。可执行性得分采用从 "1 "到 "4 "的分级系统。1 "表示完全失败, "2 "表示可执行代码, "3 "表示基本满足预期工作流程, "4 "表示完全符合预期。

| 任务         | AutoGPT | LangChain | AgentVerse | 聊天开发工具 | MetaGPT |
|------------|---------|-----------|------------|--------|---------|
| 快乐小鸟       | 1       | 1         | 1          | 2      | 3       |
| 坦克大战游戏     | 1       | 1         | 1          | 2      | 4       |
| 2048 游戏    | 1       | 1         | 1          | 1      | 4       |
| 蛇游戏        | 1       | 1         | 1          | 3      | 4       |
| 碎砖游戏       | 1       | 1         | 1          | 1      | 4       |
| Excel 数据处理 | 1       | 1         | 1          | 4      | 4       |
| CRUD 管理    | 1       | 1         | 1          | 2      | 4       |
| 平均得分       | 1.0     | 1.0       | 1.0        | 2.1    | 3.9     |

表 5: **使用不同 LLM 作为代理后端, MetaGPT 在 SoftwareDev 上的性能。**

| 模型                           | 开放源代码 | 时间 (/秒) | # 行   | 可执行性 | 修订  |
|------------------------------|-------|---------|-------|------|-----|
| MetaGPT (含 GPT-3.5)          | ■     | 75.18   | 161.6 | 2.8  | 2.4 |
| MetaGPT (含 GPT-4)            | ■     | 552.94  | 178.2 | 3.8  | 1.2 |
| MetaGPT (含 Deepseek 编码器 33B) | ✓     | 1186.20 | 120.2 | 1.4  | 2.6 |

**指令级别（高级指令与详细指令）的影响** 人类初始输入的级别差异是否会对性能结果产生重大影响？例如

1. **高级提示**: 创建一个碎砖游戏。
2. **详细提示**: 创建一个砖块闯关游戏。在碎砖游戏中, 玩家通常要控制屏幕下方的球拍, 将球弹向砖墙。目标是用球击中所有砖块, 将其击碎。

为了研究这方面的问题, 我们还进行了其他实验: 我们从 Soft-wareDev 中选择了 5 个任务, 并为它们编写了详细的提示。以下是实验结果:

表 6: **教学水平的影响**。可执行性采用从 "1 "到 "4 "的评分系统。1 "表示完全失败, "2 "表示可运行代码, "3 "表示基本符合预期的工作流程, "4 "表示完全符合预期。

| 模型  | # 字数 | 时间(/秒) | 代币用量    | # 行数  | 可执行性 | 生产率   | 返修率 |
|-----|------|--------|---------|-------|------|-------|-----|
| 高级别 | 13.2 | 552.9  | 28384.2 | 178.2 | 3.8  | 163.8 | 1.2 |
| 详细  | 42.2 | 567.8  | 29657.0 | 257.0 | 4.0  | 118.0 | 1.6 |

我们观察到: 详细的提示会带来更好的软件项目, 但由于需求和功能更清晰, 生产率会更低; 而简单的输入仍然可以使用 MetaGPT 生成足够好的软件, 可执行性评级为 3.8, 与详细提示的情况相当。(请注意, 生产率 = 代币使用量/总代码行数。该比率越低越好)。

**GPT 变体在 HumanEval 基准中的性能** 我们将 GPT-4 的 67% Hu- manEval 分数 (OpenAI, 2023 年) 作为基线, 承认它在 HumanEval 基准中的认可度。我们进一步扩展到 GPT-4 (gpt-4-0613) 和 GPT-3.5- Turbo (gpt-3.5-turbo-0613)在各种条件下的实验 (五次), 以评估性能。(A) 我们根据 HumanEval 中的提示直接调用 OpenAI API。(B) 我们调用 OpenAI API 并在响应中使用 regex 解析代码。(C) 我们添加了额外的系统提示, 然后调用 OpenAI API。提示内容为 "您是一个只能用 Python 代码 (而非英文) 进行响应的人工智能。

您将

由用户给出函数签名及其 docstring。请写出你的完整实现（重述函数签名）。如表 7 所示，在 HumanEval 数据集上，GPT-4 对提示语、代码分析器和后处理结果更为敏感。在没有提示语的情况下，GPT-3.5-Turbo 很难返回正确的完成代码。

表 7: GPT 模型在 HumanEval 上的性能。使用不同设置的 gpt-4-0613 和 gpt-3.5-turbo-0613 进行了五次实验。

| 设置 | 模型                 | 1     | 2     | 3     | 4     | 5     | 平均值   | 标准    |
|----|--------------------|-------|-------|-------|-------|-------|-------|-------|
| A  | gpt-4-0613         | 0.732 | 0.707 | 0.732 | 0.713 | 0.738 | 0.724 | 0.013 |
| A  | GPT-3.5-TURBO-0613 | 0.360 | 0.366 | 0.360 | 0.348 | 0.354 | 0.357 | 0.007 |
| B  | gpt-4-0613         | 0.787 | 0.811 | 0.817 | 0.829 | 0.817 | 0.812 | 0.016 |
| B  | GPT-3.5-TURBO-0613 | 0.348 | 0.354 | 0.348 | 0.335 | 0.348 | 0.346 | 0.007 |
| C  | gpt-4-0613         | 0.805 | 0.805 | 0.817 | 0.793 | 0.780 | 0.800 | 0.014 |
| C  | GPT-3.5-TURBO-0613 | 0.585 | 0.567 | 0.573 | 0.579 | 0.579 | 0.577 | 0.007 |

**定性结果** 图 11 和图 12 展示了架构师代理设计复杂推荐系统的成果。这些图展示了全面的系统界面设计和程序调用流程。后者对于创建复杂的自动系统至关重要。强调这种分工在开发自动化软件框架中的重要性至关重要。

## D 局限性和伦理问题

### D.1 限制

**系统方面** 目前，我们的系统还不能完全满足用户界面和前端等特定场景的需求，因为我们还没有将此类代理和多模态工具纳入其中。此外，尽管在同类框架中生成的代码量最多，但要满足现实世界中应用的多样化和复杂性要求仍具有挑战性。

**人类用户方面** 用户面临的一个主要挑战是中断每个代理的运行过程，或为每个代理设置起始运行点（检查点）。

### D.2 道德问题

**失业与技能过时** MetaGPT 让更多人能够使用自然语言编程，从而让工程师更容易上手。多年来，编程语言从打孔卡发展到汇编、C、Java、Python，再到现在的自然语言。因此，人类越来越精通编程，增加了对编程相关职位的需求。此外，使用自然语言编程可能会大大简化学习曲线，使更多人能够接触到编程。

**透明度和问责制** MetaGPT 是一个开源框架，可通过自然语言促进多个代理之间的主动交流。人类可以通过最高级别的控制来启动、服从和停止运行。它提供对自然语言的实时解释和操作，并显示在屏幕和日志上，确保透明度。MetaGPT 增强了“自然语言编程”能力，而人类工程师则是用户并对结果负责。

**隐私和数据安全** MetaGPT 在本地运行，确保用户数据的隐私和安全。它不收集用户数据。对于与第三方 LLM（如 OpenAI 的 LLM）的交互，我们鼓励用户参考各自的隐私政策（如 OpenAI 隐私政策）。不过，我们提供开源 LLM 作为后端。

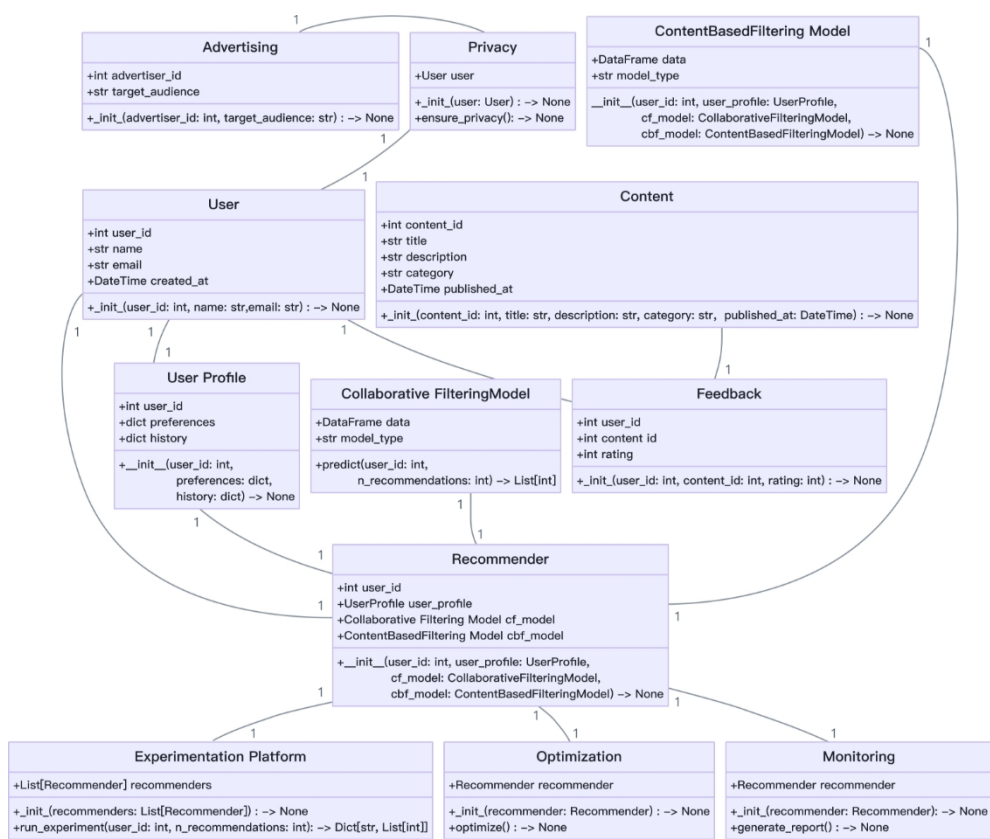


图 11: "推荐引擎开发"的系统界面设计是由架构师代理生成的（放大后可看得更清楚）。

## E 更多讨论

### E.1 根深蒂固的挑战

MetaGPT 还以其独特的设计减轻或解决了这些挑战：

**有效利用语境** 目前存在两个子挑战。第一，准确展开简短的自然语言描述，消除歧义。第二，在冗长的上下文中保持信息的有效性，使 LLM 能够集中精力处理相关数据，而不会分散注意力。

**减少幻觉** 使用 LLM 生成整个软件程序会面临代码幻觉问题--包括功能实现不完整、依赖关系缺失以及潜在的未被发现的错误，这些问题可能更为严重。由于任务定义模糊不清，LLM 在软件生成方面往往举步维艰。专注于细化任务（如需求分析和软件包选择）可以提供有指导性的思维，而这正是 LLM 在解决广泛任务时所缺乏的。

### E.2 信息超载

在 MetaGPT 中，我们使用全局消息池和订阅机制来解决 "信息过载" 问题，"信息过载" 指的是接收过多或不相关信息的问题。这个问题取决于具体的应用。MetaGPT 采用消息池来简化通信，确保效率。此外，订阅机制还能过滤掉不相关的上下文，提高信息的相关性和实用性。这种设计在软

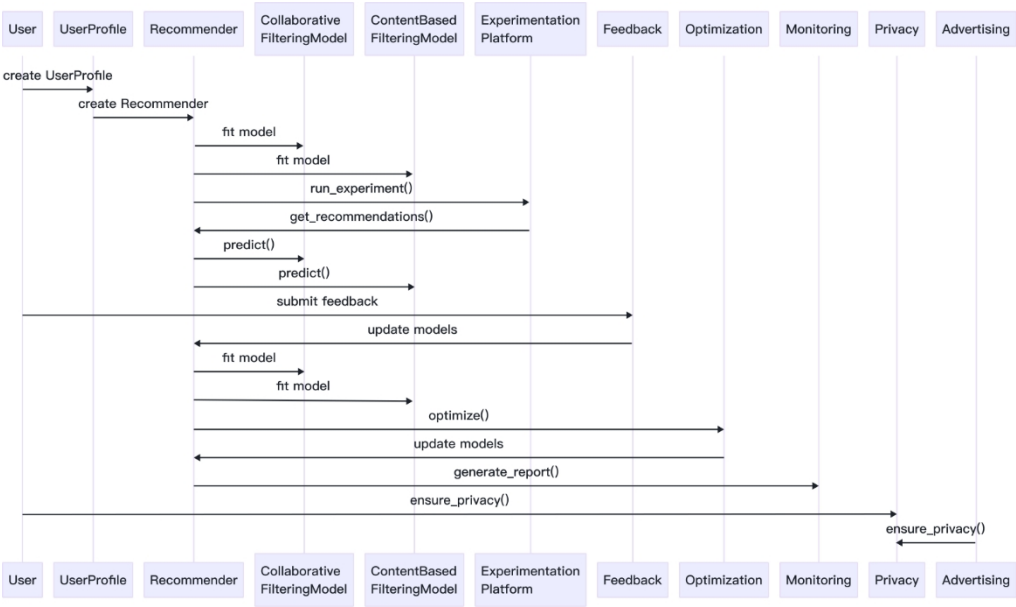


图 12: 推荐引擎开发"生成的程序调用流程  
建筑师代理 (放大查看)。

器皿设计方案和标准操作程序 (SOP)，其中有效沟通至关重要。

表 8：软件开发数据集示例。

| 任务 ID | 任务         | 提示                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0     | 蛇游戏        | 创建一个蛇形游戏。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 1     | 碎砖游戏       | 创建一个碎砖游戏。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 2     | 2048 游戏    | 为网络创建 2048 游戏。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 3     | 快乐小鸟游戏     | 为《Flappy Bird》编写 p5.js 代码，在游戏中控制一只黄色的小鸟连续飞行。小鸟在一系列绿色管道之间不停地飞行。每次点击鼠标左键，小鸟就会扇动翅膀。如果小鸟掉到地上或撞到管道上，您就输了。这个游戏会无限期地进行下去，直到你输为止；你越往前走，就越能得到分数。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 4     | 坦克大战游戏     | 创建一个坦克大战游戏。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 5     | Excel 数据处理 | 基于 streamlit 和 pandas 编写 excel 数据处理程序。该程序屏幕首先显示了一个 excel 文件上传按钮。上传 excel 文件后，使用 pandas 显示其数据内容。该程序要求简洁、易于维护、设计不过分。程序使用 streamlit 处理网页屏幕显示，使用 pandas 处理 excel 读取和显示即可。请确保其他人可以直接执行，而无需引入额外的软件包。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 6     | CRUD 管理    | Write a management program based on the crud addition, deletion, modification, update, and query. The program should be able to handle client business entity authentication and query processing. Client needs to save the following information: name, birthday, age, gender, and phone number. These data are stored in client.db, and it is necessary to judge whether the client table exists. If it does not exist, it is necessary to create the client table. Query by name, delete is also like this. The program requires simplicity, ease of maintenance, and design that is not excessive. The screen is implemented through streamlit and sqlite, without introducing other additional software packages. |
| 7     | 音乐转录器      | 开发将乐谱转录为数字格式的程序；提供-- 音乐转录器。<br>通过涉及音高和时间切片的信号处理，从音频中获取无差错转录的符号化乐谱情报，然后训练神经网络，以运行 "Onset Detected CWT"（起始检测 CWT），用递归神经网络（Recursive Neural Network）将扫描图转换为染色体解码。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 8     | 自定义新闻发布稿   | 创建自定义新闻发布稿；开发 Python 脚本以提取新闻发布稿。<br>从外部来源（如社交媒体）获取有关公司新闻的信息；提取更新间隔数据库中的最新变化。该程序应使用可定制的选项创建新闻稿，并将稿件导出为 PDF、NYTimes API JSON、带有内部固定字符长度元数据链接的媒体格式。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 9     | 五子棋游戏      | 使用 Python 实现一个五子棋游戏，并加入一个人工智能对手<br>难度各不相同。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 10    | 天气仪表盘      | 创建一个 Python 程序，开发一个交互式天气仪表盘。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

表 9: SoftwareDev 上不含反馈的纯 MetaGPT 的其他结果。计算了 70 项任务的平均值 (Average), 其中包括随机抽取的 10 项任务。#"表示 "数量", "ID "表示 "任务 ID"。

| ID  | 代码统计  |        |            | 文件统计   |        |            | 成本统计     |         |        | 修订成本   | 代码可执行性 #代                                     |      |
|-----|-------|--------|------------|--------|--------|------------|----------|---------|--------|--------|-----------------------------------------------|------|
|     | 码文件 # | 代码行数 # | 每个代码文件行数 # | 文档文件 # | 文档行数 # | 每个文档文件行数 # | 提示标记 #   | 完成标记 #  | 时间成本   | 金钱成本   |                                               |      |
| 0   | 5.00  | 196.00 | 39.20      | 3.00   | 210.00 | 70.00      | 24087.00 | 6157.00 | 582.04 | \$1.09 | 1.类型错误                                        | 4    |
| 1   | 6.00  | 191.00 | 31.83      | 3.00   | 230.00 | 76.67      | 32517.00 | 6238.00 | 566.30 | \$1.35 | 1.类型错误                                        | 4    |
| 2   | 3.00  | 198.00 | 66.00      | 3.00   | 235.00 | 78.33      | 21934.00 | 6316.00 | 553.11 | \$1.04 | 1. 无 @app.route('/')                          | 3    |
| 3   | 5.00  | 164    | 32.80      | 3.00   | 202.00 | 67.33      | 22951.00 | 5312.00 | 481.34 | \$1.01 | 1.PNG 文件错误 2. 编译错误修复                          | 2    |
| 4   | 6.00  | 203.00 | 33.83      | 3.00   | 210.00 | 70.00      | 30087.00 | 6567.00 | 599.58 | \$1.30 | 1. PNG 文件缺少 2.修复了许多错误 3. pygame.surface 无法初始化 | 3    |
| 5   | 6.00  | 219.00 | 36.50      | 3.00   | 294.00 | 96.00      | 35590.00 | 7336.00 | 585.10 | \$1.51 | 1. 受抚养人ror 2.模块未找到错误                          | 4    |
| 6   | 4.00  | 73.00  | 18.25      | 3.00   | 261.00 | 87.00      | 25673.00 | 5832.00 | 398.83 | \$0.90 | 0                                             | 4    |
| 7   | 4.00  | 316.00 | 79.00      | 3.00   | 332.00 | 110.67     | 29139.00 | 7104.00 | 435.83 | \$0.92 | 0                                             | 4    |
| 8   | 5.00  | 215.00 | 43.00      | 3.00   | 301.00 | 100.33     | 29372.00 | 6499.00 | 621.73 | \$1.27 | 1. tensorflow ver-模型训练方法未落实                   | 2    |
| 9   | 5.00  | 215.00 | 43.00      | 3.00   | 270.00 | 90.00      | 24799.00 | 5734.00 | 550.88 | \$1.27 | 1. 依赖性 error 2.URL 403 error                  | 3    |
| 10  | 3.00  | 93.00  | 31.00      | 3.00   | 254.00 | 84.67      | 24109.00 | 5363.00 | 438.50 | \$0.92 | 1. 依赖性缺少主要功能。                                 | 4    |
| 平均值 | 4.71  | 191.57 | 42.98      | 3.00   | 240.00 | 80.00      | 26626.86 | 6218.00 | 516.71 | \$1.12 | 0.51 （只考虑得 2 分、3 分或 4)                        | 3.36 |